Puppet Base Course

Day 3

Structuring Code, Version Control & Quality Assurance

Day 3 — Agenda

Structuring Puppet Code

- Patterns for building modules — the component module
- The Roles and Profiles pattern — the industry standard
- Patterns for structuring manifests within a module

Version Control for Puppet

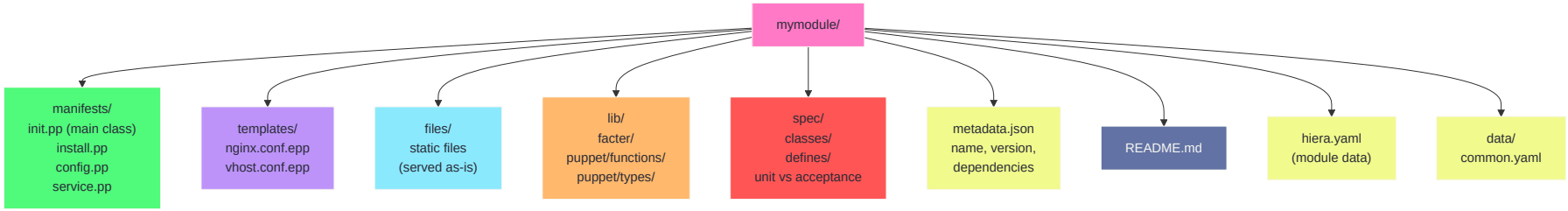
- Why version control is non-negotiable for configuration management
- Introduction to Git — the essentials for Puppet practitioners
- The control repository — your Puppet infrastructure's single source of truth
- r10k: environments from branches, Puppetfiles, and deployment

Quality Assurance

- PDK — the Puppet Development Kit
- Testing modules with rspec-puppet
- rspec-puppet in practice: classes, defined types, and parameters
- Acceptance testing overview: Litmus and serverspec
- Code review for Puppet — what to look for
- puppet-lint and code style

Part 1 — Structuring Puppet Code

Module Structure



The Module Directory Layout

```
mymodule/
├── manifests/
│   ├── init.pp          ← main class (class mymodule)
│   ├── install.pp       ← class mymodule::install
│   ├── config.pp        ← class mymodule::config
│   └── service.pp       ← class mymodule::service
├── templates/
│   ├── myapp.conf.epp
│   └── vhost.conf.epp
├── files/
│   └── static-file.conf
├── lib/
│   ├── facter/
│   │   └── myapp_version.rb
│   └── puppet/functions/
│       └── mymodule/validate_input.rb
├── spec/
│   └── classes/
│       └── init_spec.rb
├── data/
│   └── common.yaml
├── hiera.yaml
├── metadata.json
└── README.md
```

`metadata.json` — Module Identity

Every module must have a `metadata.json`:

```
{
  "name": "myorg-nginx",
  "version": "1.2.0",
  "author": "myorg",
  "license": "Apache-2.0",
  "summary": "Manage nginx web server",
  "source": "https://github.com/myorg/puppet-nginx",
  "project_page": "https://github.com/myorg/puppet-nginx",
  "issues_url": "https://github.com/myorg/puppet-nginx/issues",
  "dependencies": [
    { "name": "puppetlabs-stdlib", "version_requirement": ">= 6.0.0 < 10.0.0" }
  ],
  "requirements": [
    { "name": "puppet", "version_requirement": ">= 7.0.0 < 9.0.0" }
  ],
  "operatingsystem_support": [
    { "operatingsystem": "Ubuntu", "operatingsystemrelease": [ "20.04", "22.04" ] },
    { "operatingsystem": "RedHat", "operatingsystemrelease": [ "8", "9" ] }
  ]
}
```

The Component Module Pattern

A **component module** manages **one technology** completely:

```
# class nginx — public API
class nginx (
  Integer $worker_processes = 2,
  Array   $listen_ports     = [80],
  Boolean $enable_gzip      = true,
) {
  contain nginx::install
  contain nginx::config
  contain nginx::service

  Class['nginx::install']
    -> Class['nginx::config']
    ~> Class['nginx::service']
}
```

The user of the module only touches `class nginx {}` and its parameters. The sub-classes are private implementation details.

Component Module — Install Class

```
# manifests/install.pp
class nginx::install {
  package { 'nginx':
    ensure => $nginx::package_ensure,
  }
}
```

Component Module — Config Class

```
# manifests/config.pp
class nginx::config {
  file { '/etc/nginx/nginx.conf':
    ensure  => file,
    content => epp('nginx/nginx.conf.epp', {
      worker_processes => $nginx::worker_processes,
      enable_gzip      => $nginx::enable_gzip,
    }),
  }
```

Puppet Forge — Reusing Community Modules

You don't have to write everything yourself.

```
forge.puppet.com
```

Find certified, community-vetted modules for:

Use case	Module
Apache	`puppetlabs-apache`
MySQL / MariaDB	`puppetlabs-mysql`
PostgreSQL	`puppetlabs-postgresql`
Users & groups	`puppetlabs-accounts`
NTP	`puppetlabs-ntp` or `saz-ntp`
Firewall (iptables/nftables)	`puppetlabs-firewall`

The Roles and Profiles Pattern

The most important architectural pattern for real-world Puppet:

```
node 'web01.example.com'
include role::webserver
```

Why Roles and Profiles?

The problem with raw component modules on nodes:

```
# site.pp — fragile, repetitive, impossible to maintain
node 'web01' {
  class { 'nginx': worker_processes => 4 }
  class { 'php-fpm': version => '8.2' }
  class { 'mysql': root_password => ... }
  # 40 more class declarations...
}
```

With Roles and Profiles:

```
# site.pp — clean and expressive
node 'web01.example.com' {
  include role::webserver
}
```


Everything else lives in Hiera and profiles. Node classification becomes **one line**.

Profiles — Technology Stacks

A **profile** wraps one or more component modules, passing Hiera data as parameters:

```
# modules/profile/manifests/nginx.pp
class profile::nginx {
  class { 'nginx':
    worker_processes => lookup('profile::nginx::worker_processes', Integer, 'first', 2),
    listen_ports     => lookup('profile::nginx::listen_ports', Array, 'unique', [80]),
    enable_gzip      => lookup('profile::nginx::enable_gzip', Boolean, 'first', true),
  }

  # Site-specific vhosts from Hiera
  $vhosts = lookup('profile::nginx::vhosts', Hash, 'hash', {})
  $vhosts.each |$name, $config| {
    nginx::vhost { $name:
      * => $config,
    }
  }
}
```

Roles — Business Roles

A **role** describes **what a node is** by composing profiles:

```
# modules/role/manifests/webserver.pp
class role::webserver {
  include profile::base        # baseline: users, NTP, monitoring agent
  include profile::nginx       # web server
  include profile::php_fpm     # PHP runtime
  include profile::app_deploy  # application deployment
  include profile::logging     # log shipper
}

# modules/role/manifests/database.pp
class role::database {
  include profile::base
  include profile::mysql
  include profile::backups
  include profile::monitoring::db
}
```

Roles and Profiles — The Rules

Layer	Rules
Roles	Only `include` profiles. No resources. No Hiera lookups. One role per node.
Profiles	Use component modules. All configuration from Hiera. May include other profiles.
Component modules	Manage one technology. Accept all configuration as parameters. Forge or custom.

> This separation makes your infrastructure readable, testable, and maintainable at any scale.

Structuring Manifests — The `contain` vs `include` Choice

```
class nginx {
  # contain: establishes ordering boundaries
  # resources in nginx::install won't bleed past the nginx class boundary
  contain nginx::install
  contain nginx::config
  contain nginx::service

  # Chain with arrows
  Class['nginx::install']
    -> Class['nginx::config']
    ~> Class['nginx::service']
}
```

> Use `contain` inside a module to keep ordering predictable for callers.
> Use `include` at the role/profile level to assign classes to nodes.

Module Data — Module-Specific Hiera

Modules can ship their own Hiera defaults, which are **lower priority** than environment data:

```yaml
# modules/nginx/hiera.yaml
---
version: 5
defaults:
  datadir: data
  data_hash: yaml_data
hierarchy:
  - name: "OS family"
    path: "os/%{facts.os.family}.yaml"
  - name: "Common"
    path: "common.yaml"
```

```yaml
# modules/nginx/data/common.yaml
nginx::worker_processes: 2
nginx::enable_gzip: true
```

Part 2 — Version Control and r10k

Why Version Control for Puppet Code?

Without version control:

- Editing manifests directly on the Puppet Server — one typo takes down all agents
- No rollback when a change breaks production
- No review process — anyone can change anything
- No audit trail — who changed what, and why?

With Git:

- Every change is a commit — full audit trail
- Review via pull requests before code reaches production
- Roll back a breaking change instantly
- Multiple environments from branches (production, staging, feature)

Git — The Essentials for Puppet Practitioners

```
# Initialise a new repository
git init

# Clone an existing repository (control repo)
git clone git@github.com:myorg/puppet-control.git

# Check status
git status

# Stage and commit changes
git add modules/profile/manifests/nginx.pp
git commit -m "profile::nginx: add gzip configuration"

# Branch for a new feature
git checkout -b feature/add-redis-profile

# Push to remote
git push origin feature/add-redis-profile

# Merge via pull request (GitHub/GitLab UI), then:
git checkout production && git pull
```

The Control Repository

The **control repository** is the single source of truth for your entire Puppet code base:

```
control-repo/
├── environment.conf          ← environment configuration
├── hiera.yaml                ← global Hiera hierarchy
├── Puppetfile                ← module declarations (r10k)
├── data/
│   ├── common.yaml
│   ├── os/
│   └── nodes/
├── manifests/
│   └── site.pp               ← node classification
└── modules/
    ├── profile/              ← your profiles
    └── role/                 ← your roles
```

> One `git push` to the `production` branch updates production. One push to `staging` updates staging. **Git branches = Puppet environments.**

The Puppetfile

The **Puppetfile** declares all external module dependencies:

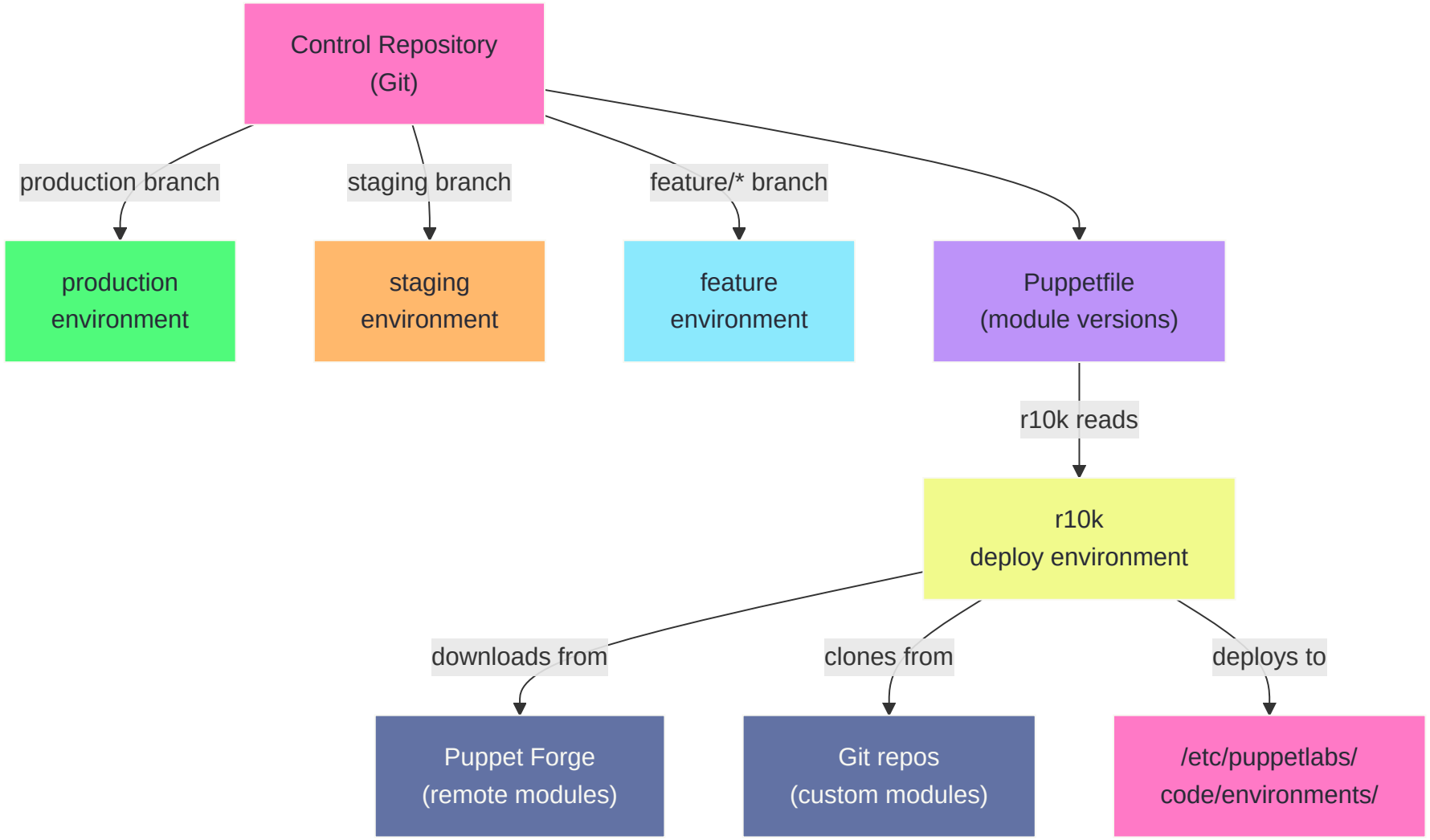
```
# Puppetfile
forge "https://forgeapi.puppet.com"

# Forge modules — pin exact versions for reproducibility
mod 'puppetlabs-stdlib',  '9.4.1'
mod 'puppetlabs-apache',  '12.1.0'
mod 'puppetlabs-mysql',   '15.0.0'
mod 'puppetlabs-ntp',     '10.0.0'
mod 'puppetlabs-firewall', '7.0.0'

# Git-hosted modules — specify branch or tag
mod 'myorg-nginx',
  git: 'git@github.com:myorg/puppet-nginx.git',
  tag: 'v2.3.0'

mod 'myorg-app',
  git: 'git@github.com:myorg/puppet-app.git',
  branch: 'main'
```

r10k — Environments from Branches



Installing and Configuring r10k

```
# Install r10k (on the Puppet Server)
gem install r10k
# or: /opt/puppetlabs/puppet/bin/gem install r10k
```

```yaml
# /etc/puppetlabs/r10k/r10k.yaml
---
cachedir: '/var/cache/r10k'

sources:
  control:
    remote: 'git@github.com:myorg/puppet-control.git'
    basedir: '/etc/puppetlabs/code/environments'
```

```
# Deploy all environments (branches → directories)
r10k deploy environment -pv

# Deploy a single environment (e.g., after a push)
r10k deploy environment production -pv
```

r10k Deployment Workflow

```
Developer:
  git checkout -b feature/new-profile
  # ... write code ...
  git push origin feature/new-profile

Puppet Server:
  r10k deploy environment feature_new_profile
  # Creates /etc/puppetlabs/code/environments/feature_new_profile/

Test:
  puppet agent --test --environment feature_new_profile

Review + Merge:
  # Pull request reviewed and merged to production branch

Deploy:
  r10k deploy environment production -pv
  # Updates production environment from the production branch
```

Automating r10k with a Webhook

Trigger r10k automatically on every `git push`:

```
# Install the webhook gem
gem install puppet_webhook

# Example webhook endpoint
# POST /api/v1/r10k/environment  ->  r10k deploy environment <branch>
```

Alternatively, use CI/CD pipelines (Jenkins, GitLab CI, GitHub Actions):

```yaml
# .github/workflows/deploy.yml
on:
  push:
    branches: [production, staging]
jobs:
  deploy:
    runs-on: self-hosted
    steps:
      - run: r10k deploy environment ${{ github.ref_name }} -p
```

Part 3 — Quality Assurance

The PDK — Puppet Development Kit

The PDK is the official scaffold and test tool for Puppet modules:

```
# Install PDK
# https://www.puppet.com/try-puppet/puppet-development-kit/

# Create a new module scaffold
pdk new module mymodule

# Create a new class
pdk new class mymodule::install

# Create a new defined type
pdk new defined_type mymodule::vhost

# Validate syntax and style
pdk validate

# Run unit tests
pdk test unit

# Run unit tests and show coverage
```

Testing Layers

Lint / Style

puppet-lint, PDK validate

Syntax, style, best practice

fast feedback

Unit Tests

rspec-puppet

Test catalog compilation,

resource declarations,

class parameters

rspec-puppet — Unit Testing

rspec-puppet compiles the Puppet catalog in memory and asserts on its contents — no actual nodes needed:

```ruby
# spec/classes/init_spec.rb
require 'spec_helper'

describe 'nginx' do
  # Test default parameters
  context 'with default parameters' do
    it { is_expected.to compile.with_all_deps }
    it { is_expected.to contain_package('nginx').with_ensure('installed') }
    it { is_expected.to contain_service('nginx').with_ensure('running') }
    it { is_expected.to contain_file('/etc/nginx/nginx.conf') }
  end

  # Test parameter override
  context 'with enable_gzip => false' do
    let(:params) { { enable_gzip: false } }
    it { is_expected.to compile }
    it { is_expected.to contain_file('/etc/nginx/nginx.conf')
```

rspec-puppet — Testing with Facts and OS Context

```ruby
describe 'nginx' do
  on_supported_os.each do |os, os_facts|
    context "on #{os}" do
      let(:facts) { os_facts }

      it { is_expected.to compile.with_all_deps }

      if os_facts[:os]['family'] == 'Debian'
        it { is_expected.to contain_package('nginx') }
        it { is_expected.to contain_service('nginx')
          .with_ensure('running')
          .with_enable(true) }
      end
    end
  end
end
```

> `on_supported_os` iterates over all OS/version combinations declared in `metadata.json` — tests run on all supported platforms automatically.

rspec-puppet — Testing Defined Types

```
describe 'nginx::vhost' do
  let(:title) { 'my-site' }

  context 'with minimum parameters' do
    let(:params) do
      { servername: 'www.example.com' }
    end

    it { is_expected.to compile.with_all_deps }

    it {
      is_expected.to contain_file('/etc/nginx/sites-available/my-site')
        .with_ensure('file')
    }

    it {
      is_expected.to contain_file('/etc/nginx/sites-enabled/my-site')
        .with_ensure('link')
        .with_target('/etc/nginx/sites-available/my-site')
    }
  end

  context 'with invalid port' do
    let(:params) { { servername: 'x.com', port: 99999 } }
    it { is_expected.to compile.and_raise_error(/expects an Integer\[1, 65535\]/) }
  end
end
```

rspec-puppet — Testing Relationships

```ruby
describe 'nginx' do
  let(:facts) { { os: { family: 'Debian', name: 'Ubuntu' } } }

  it { is_expected.to compile.with_all_deps }

  # Test that the config file notifies the service
  it {
    is_expected.to contain_file('/etc/nginx/nginx.conf')
      .that_notifies('Service[nginx]')
  }

  # Test that the service requires the package
  it {
    is_expected.to contain_service('nginx')
      .that_requires('Package[nginx]')
  }
end
```

The `spec_helper.rb`

```ruby
# spec/spec_helper.rb
require 'puppetlabs_spec_helper/module_spec_helper'
require 'rspec-puppet-facts'

include RspecPuppetFacts

RSpec.configure do |config|
  config.mock_with :rspec
end
```

Gemfile for Testing

```ruby
# Gemfile
source 'https://rubygems.org'

gem 'puppet',                ENV['PUPPET_VERSION'] || '~> 8.0'
gem 'puppetlabs_spec_helper', '~> 7.0'
gem 'rspec-puppet'           '~> 4.0'
```

puppet-lint — Code Style

puppet-lint checks your code against the Puppet Language Style Guide:

```
# Run puppet-lint on all manifests
puppet-lint manifests/

# Run via PDK
pdk validate puppet

# Common fixable checks
puppet-lint --fix manifests/init.pp
```

Common puppet-lint Checks

Check	Bad	Good
Trailing whitespace	`package { 'nginx':`	`package { 'nginx':`
Double-quoted strings	`ensure => "present"`	`ensure => present`
Arrow alignment	Misaligned `=>`	Aligned `=>` in a block
Variable quoting	`$var` in string	`"${var}"`
Class parameter docs	Missing `@param`	Document all params
Resource title quoting	`package { nginx: }`	`package { 'nginx': }`

```
# Disable a specific check inline
# lint:ignore:double_quoted_strings
$string = "I need double quotes here"
# lint:endignore
```

Acceptance Testing — Litmus and serverspec

Unit tests verify the catalog compiles and contains the right resources.

Acceptance tests verify the code actually works on a real system.

```
# Provision test containers (Docker)
pdk bundle exec rake 'litmus:provision[docker, ubuntu:22.04]'

# Install Puppet agent on containers
pdk bundle exec rake litmus:install_agent

# Install the module
pdk bundle exec rake litmus:install_module

# Run acceptance tests
pdk bundle exec rake litmus:acceptance:localhost
```

serverspec Example

```ruby
# spec/acceptance/nginx_spec.rb
require 'spec_helper_acceptance'

describe 'nginx class' do
  it 'applies without errors' do
    pp = "class { 'nginx': }"
    apply_manifest(pp, catch_failures: true)
    apply_manifest(pp, catch_changes: true)  # idempotency check
  end

  describe package('nginx') do
    it { is_expected.to be_installed }
  end

  describe service('nginx') do
    it { is_expected.to be_running }
    it { is_expected.to be_enabled }
  end

  describe port(80) do
    it { is_expected.to be_listening }
  end

  describe file('/etc/nginx/nginx.conf') do
    it { is_expected.to be_file }
    its(:content) { is_expected.to match /worker_processes/ }
  end
end
```

Code Review for Puppet — What to Look For

Correctness

- Does the catalog compile? Any circular dependencies?
- Are resource relationships correct (ordering and refresh)?
- Are types and validation used on all parameters?

Security

- No secrets hardcoded — use Hiera eyaml or a secrets backend
- File modes are explicit and not overly permissive
- `exec` resources use absolute paths and have `creates` / `unless` / `onlyif`

Maintainability

- Does the change follow the Roles and Profiles pattern?
- Is new data in Hiera, not hardcoded in manifests?

Code Review — Red Flags

```
# ❌ Hardcoded secret
class myapp {
  $db_password = 'super-secret-123'
}

# ❌ exec without guard (runs on every agent run)
exec { 'run-migration':
  command => '/opt/app/bin/migrate',
}

# ❌ Data in code
class ntp {
  $servers = ['ntp1.example.com', 'ntp2.example.com']
}

# ❌ Ambiguous ensure — use 'present' or 'installed', not bare variable
package { $name: ensure => $ensure }
```

Code Review — Green Flags

```
# ✅ Secret from Hiera eyaml
$db_password = lookup('myapp::db_password')

# ✅ exec with guard — only runs once
exec { 'run-migration':
  command => '/opt/app/bin/migrate --env production',
  creates => '/opt/app/.migrated',
  path    => ['/opt/app/bin', '/usr/bin', '/bin'],
}

# ✅ Data in Hiera
class ntp (Array[String] $servers = []) { }

# ✅ Type-validated parameters
class myapp (
  String[1]        $app_name,
  Integer[1, 65535] $port,
  Boolean           $enable_tls = true,
) { }
```

CI/CD Pipeline for Puppet Modules

```yaml
# .github/workflows/ci.yml
name: CI

on: [push, pull_request]

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: ruby/setup-ruby@v1
        with: { ruby-version: '3.1' }
      - run: bundle install
      - run: bundle exec rake validate lint
      - run: bundle exec rake spec

  acceptance:
    runs-on: ubuntu-latest
    needs: validate
    steps:
      - uses: actions/checkout@v3
      - uses: ruby/setup-ruby@v1
      - run: bundle install
      - run: bundle exec rake litmus:acceptance
```

Putting It All Together — A Workflow

```
1. git checkout -b feature/add-redis-module
2. pdk new class profile::redis
3. Write profile::redis in manifests/redis.pp
4. Add Hiera data to data/
5. pdk validate && pdk test unit
6. Commit and push
7. Open Pull Request
8. CI runs: lint → unit tests → acceptance tests
9. Team reviews code
10. Merge to staging → r10k deploys → QA team tests
11. Merge to production → r10k deploys → monitoring watches
```

Part 4 — Bolt, PuppetDB & Enterprise Practices

Puppet Bolt — Ad-Hoc Orchestration

Bolt runs commands, scripts, and tasks **on-demand** without a Puppet agent:

```
# Run a shell command across multiple nodes (SSH)
bolt command run 'systemctl status nginx' \
  --targets web01.example.com,web02.example.com

# Same using an inventory group
bolt command run 'df -h' --targets webservers

# Upload and execute a local script
bolt script run scripts/check-db.sh --targets databases

# Run a Bolt task from an installed module
bolt task run package action=status name=nginx \
  --targets web01.example.com
```

> Bolt uses **SSH** (Linux) or **WinRM** (Windows) — no agent installation required.
> Ideal for bootstrapping, incident response, and one-off operational tasks.

Bolt Inventory

Organise targets into groups with shared connection settings:

```yaml
# inventory.yaml
config:
  ssh:
    user: deploy
    private-key: ~/.ssh/puppet_deploy_key
    host-key-check: false

groups:
  - name: webservers
    targets:
      - web01.example.com
      - web02.example.com
    config:
      ssh:
        user: www-deploy

  - name: databases
    targets:
      - db01.example.com
      - db02.example.com
    config:
```

Bolt Tasks

A task is a script with a JSON metadata sidecar:

```
# modules/mymodule/tasks/restart_service.sh
#!/bin/bash
# Parameters are passed as PT_<name> environment variables
set -e
systemctl restart "$PT_service_name"
echo "Restarted: ${PT_service_name}"
```

```
{
  "description": "Restart a named system service",
  "parameters": {
    "service_name": {
      "description": "Name of the service to restart",
      "type": "String[1]"
    }
  }
}
```

Bolt Plans — Orchestrated Workflows

Plans compose tasks, commands, and Puppet applies into multi-step workflows:

```
# modules/myapp/plans/rolling_deploy.pp
plan myapp::rolling_deploy(
  TargetSpec $targets,
  String     $version,
) {
  $targets.each |$target| {
    out::message("Deploying ${version} to ${target}")

    # Drain from load balancer
    run_task('haproxy::disable', 'lb01.example.com',
      backend => 'web', server => $target.name)

    # Deploy new version
    run_task('myapp::deploy', $target, version => $version)

    # Health-check before re-enabling
    run_task('myapp::healthcheck', $target)

    # Re-enable in load balancer
    run_task('haproxy::enable', 'lb01.example.com',
      backend => 'web', server => $target.name)
  }
```

PuppetDB — The Data Backbone

PuppetDB stores everything Puppet knows about your infrastructure:

Data	What's stored	Retention
Facts	Facter output, every node, every run	Latest per node
Catalogs	Last compiled catalog per node	Latest per node
Reports	Run outcome, changed/failed resources, logs	Configurable (default 14 d)
Resource events	Every individual resource change	With reports

```
# Check PuppetDB health
curl -s https://puppet.example.com:8081/status/v1/services \
  --cacert /etc/puppetlabs/puppet/ssl/certs/ca.pem \
  --cert   /etc/puppetlabs/puppet/ssl/certs/puppet.pem \
```

PuppetDB — `puppet query` and PQL

PQL (Puppet Query Language) lets you interrogate your entire fleet:

```
# Find all nodes running Ubuntu 22.04
puppet query "nodes { facts { name = 'os.name' and value = 'Ubuntu' }
                  and facts { name = 'os.release.full' and value = '22.04' } }"

# Find nodes where nginx service is stopped
puppet query "resources[certname] {
  type = 'Service' and title = 'nginx'
  and parameters.ensure = 'stopped' }"

# Find all nodes that changed in the last run
puppet query "reports[certname] {
  status = 'changed'
  order by receive_time desc limit 20 }"

# Find nodes that have NOT run in 24 hours (detect stale agents)
puppet query "nodes[certname] {
  latest_report_timestamp < '$(date -u -d '24 hours ago' +%Y-%m-%dT%H:%M:%SZ)' }"
```

PuppetDB — Fleet Visibility Queries

```
# Count nodes by OS — real-time OS inventory
puppet query "facts[value, count(certname)] {
  name = 'os.name'
  group by value
  order by count(certname) desc }"

# Find all nodes with a specific package installed
puppet query "resources[certname, parameters] {
  type = 'Package' and title = 'nginx'
  and parameters.ensure != 'absent' }"

# Find nodes with catalog compilation failures
puppet query "reports[certname, receive_time] {
  status = 'failed'
  order by receive_time desc }"

# Find all distinct values of a fact across the fleet
puppet query "facts[value, count(certname)] {
  name = 'datacenter'
  group by value }"
```

Puppet Strings — Self-Documenting Modules

Puppet Strings generates HTML/JSON/Markdown docs from structured comment blocks:

```
# @summary Manages the nginx web server.
#
# Installs, configures, and manages the nginx service.
#
# @example Basic usage
#   include nginx
#
# @example With custom worker count
#   class { 'nginx':
#     worker_processes => 8,
#   }
#
# @param worker_processes Number of nginx worker processes.
# @param enable_gzip Whether to enable gzip compression in responses.
# @param listen_ports Array of TCP ports nginx should listen on.
class nginx (
  Integer        $worker_processes = $facts['processors']['count'],
```

Semantic Versioning for Modules

Follow SemVer — `MAJOR.MINOR.PATCH`:

Change	Bump	Example
Breaking: param renamed/removed, OS dropped	MAJOR	1.2.3 → 2.0.0
New feature, backward-compatible	MINOR	1.2.3 → 1.3.0
Bug fix	PATCH	1.2.3 → 1.2.4

```
## CHANGELOG

## [2.0.0] - 2026-04-01
### Breaking Changes
- Renamed parameter `worker_count` → `worker_processes`
- Dropped Puppet 6 support

## [1.3.0] - 2026-03-15
### Added
```

Publishing to the Puppet Forge

```
# Validate metadata.json and build the module tarball
pdk build
# → pkg/myorg-nginx-2.0.0.tar.gz

# Publish via PDK (requires FORGE_TOKEN env var)
pdk release --forge-token "$FORGE_TOKEN"

# Or publish manually
puppet module publish ./pkg/myorg-nginx-2.0.0.tar.gz \
  --module_repository https://forgeapi.puppet.com
```

```
{
  "name": "myorg-nginx",
  "version": "2.0.0",
  "author": "myorg",
  "license": "Apache-2.0",
  "summary": "Manages the nginx web server",
  "source": "https://github.com/myorg/puppet-nginx",
  "dependencies": [
```

Multi-Datacenter r10k

```yaml
# /etc/puppetlabs/r10k/r10k.yaml
---
cachedir: '/var/cache/r10k'

sources:
  # Main control repository
  control:
    remote:  'git@github.com:myorg/puppet-control.git'
    basedir: '/etc/puppetlabs/code/environments'
    prefix:  false

  # Extra internal module repo — merged into every environment
  site_modules:
    remote:  'git@github.com:myorg/puppet-site-modules.git'
    basedir: '/etc/puppetlabs/code/modules'
    prefix:  true
```

```bash
# Deploy production simultaneously across all Puppet Servers
for puppet_server in dc1-puppet.example.com dc2-puppet.example.com; do
```

Environment Promotion Pipeline

```
feature/add-redis                          (developer branch)
        │
        ▼
   Code Review (PR)            CI: puppet-lint + rspec-puppet
        │
        ▼
     staging branch    ──────▶ r10k deploys staging environment
     (merge)                   test: puppet agent --environment staging
                               QA approval required
        │
        ▼
  production branch    ──────▶ r10k deploys production environment
  (merge + git tag)             monitoring watches rollout
```

```bash
# Trigger production deploy from CI after merge
git tag -a "v$(date +%Y.%m.%d)" -m "Production release"
git push origin --tags
# GitHub Actions: ssh puppet-server r10k deploy environment production -pv
```

Advanced rspec-puppet — Shared Examples

Avoid test duplication with `shared_examples`:

```
# spec/support/shared_examples_for_profile.rb
shared_examples 'a managed profile' do
  it { is_expected.to compile.with_all_deps }
  it { is_expected.to contain_class('profile::base') }
  it {
    is_expected.to contain_file('/etc/puppet-managed')
      .with_ensure('file')
  }
end

# Reuse in multiple profile spec files
describe 'profile::nginx' do
  it_behaves_like 'a managed profile'
  # nginx-specific tests …
end

describe 'profile::mysql' do
  it_behaves_like 'a managed profile'
  # mysql-specific tests
```

rspec-puppet — Code Coverage

Configure and enforce a minimum coverage threshold:

```ruby
# spec/spec_helper.rb
RSpec.configure do |c|
  c.after(:suite) do
    RSpec::Puppet::Coverage.report!(80)  # fail if < 80% covered
  end
end
```

```
bundle exec rake spec 2>&1 | tail -15

# Coverage report:
# Total resources:   42
# Touched resources: 36
# Resource coverage: 85.71%
#
# Untouched resources:
#   File[/etc/myapp/emergency.conf]
#   Exec[post-install-hook]
```

rspec-puppet — Testing Hiera Data Integration

```ruby
describe 'profile::ntp' do
  # Provide Hiera data in tests just like the real hierarchy
  let(:hiera_config) { 'spec/fixtures/hiera.yaml' }

  context 'with production NTP servers' do
    let(:facts) { { environment: 'production' } }
    let(:hiera_data) do
      { 'profile::ntp::servers' => ['ntp.prod.example.com'] }
    end

    it {
      is_expected.to contain_file('/etc/ntp.conf')
        .with_content(/ntp\.prod\.example\.com/)
    }
  end

  context 'with default NTP servers' do
    it {
      is_expected.to contain_file('/etc/ntp.conf')
        .with_content(/pool\.ntp\.org/)
    }
  end
end
```

Puppet Enterprise vs. Open Source

Feature	Open Source	Puppet Enterprise
Agent/Server architecture	✅	✅
Puppet DSL, Hiera, Forge	✅	✅
Web console (GUI)	❌	✅
Node groups + rules-based classification	❌	✅
Role-based access control (RBAC)	❌	✅
Orchestrator (mass targeted applies)	❌	✅
CD4PE (pipeline + impact analysis)	❌	✅
Supported Bolt integration	Limited	✅
Official support + SLA	Community	Perforce SLA

Puppet Enterprise — Node Groups and Classification

PE replaces `site.pp` with a GUI-driven classification model:

```
Node Groups (PE Console)
├── All Nodes
│   └── PE Infrastructure
│       ├── PE Primary Server
│       └── PE PuppetDB
└── Production Environment
    ├── Webservers          → role::webserver
    │   Rule: fact["role"] = "webserver"
    ├── Databases           → role::database
    │   Rule: fact["role"] = "database"
    └── Monitoring Servers → role::monitoring
        Rule: fact["datacenter"] = "dc1"
          AND fact["role"] = "monitoring"
```

> Node Groups use **fact-based rules** (dynamic) or **pinned nodes** (static). Rules are evaluated per agent run — nodes move between groups automatically as

Day 3 — Summary

Concept	Key takeaway
Component modules	Manage one technology; `contain` sub-classes with arrows
Roles and Profiles	Roles = node identity; Profiles = technology stacks; all data in Hiera
Puppet Forge	Reuse certified, maintained modules
Control repository	Git branches = Puppet environments
Puppetfile	Declares all module versions — like a lockfile
r10k	Deploys environments from branches; installs Puppetfile modules
PDK	Standard scaffold and test runner for Puppet modules

Course Summary — What You've Learned

Day	Topics
Day 1	CM concepts, Puppet architecture, AIO install, certificates, Puppet DSL basics: resources, ordering, conditionals, classes, defined types
Day 2	Iterators, facts, custom facts, ERB/EPP templates, Hiera hierarchy, APL, merge strategies, eyaml
Day 3	Module patterns, Roles and Profiles, control repo, Puppetfile, r10k, PDK, rspec-puppet, acceptance testing, code review

Where to Go from Here

Topic	Resource
Puppet documentation	`puppet.com/docs`
Puppet Forge	`forge.puppet.com`
rspec-puppet docs	`rspec-puppet.com`
PDK docs	`puppet.com/docs/pdk`
Puppet Style Guide	`puppet.com/docs/puppet/latest/style_guide.html`
Voxpupuli	Open-source community modules — `voxpupuli.org`
Puppet Slack	`slack.puppet.com`

> The best next step: build a control repo for your organisation, convert one manual server to Puppet, and grow from there.